Téma 4: Doplnenie ciest

📖 Čo sa tu naučím

- Prečo Apache log nezachytáva všetky stránky (browser cache, sieťové problémy, multi-tab)
- Princíp Path Completion z mapy webu (parent / ReferencePage)
- Algoritmus doplnenia: iterácia párov, kontrola sedenia, lookup rodiča
- Rozdiel medzi 4 metódami identifikácie sedení a ich očakávanými počtami doplnení
- Edge cases: refresh, hranica sedenia, posledný riadok, prázdny root

🔑 Najdôležitejšie vzorce a koncepty

> **Z prednášky 8 (Fázy procesu — Príprava dát, Doplnenie ciest):** Po identifikácii sedení nasleduje fáza **Doplnenie záznamov logovacieho súboru**. Server-side log nezachytáva všetky reálne navštívené stránky — niektoré sa zobrazia z **browser cache** alebo cez stratené/preplietené requesty. Pomocou **mapy webu** (URL → Odkazujúca stránka, t. j. `parent`) rekonštruujeme chýbajúce medzihopy: ak aktuálna navigácia obsahuje pár (`stránka B → stránka X`), ale v mape je `parent[X] = A`, pričom `B` v mape ako rodič `X` nie je, vložíme medzi ne kanonického rodiča.

Hlavná podmienka doplnenia:

```
if url_from not in map_paths and url_to in map_paths:
    ref_page = parent.get(url_to)
    if ref_page and ref_page != url_from:
        # vlož doplnený riadok s URL = ref_page
```

Štruktúry pre rýchlosť: - `parent` (dict) → O(1) lookup rodiča - `map_paths` (set) → O(1) test `in` - `new_rows` (list) → amortizované O(1) append, finálne `pd.DataFrame(new_rows)`

Očakávané počty doplnených: | Metóda | Počet | |--------|-------| | STT_Mean | ~750 | | STT_Q | ~750 | | RLength | ~850 | | h-ref | ~690 |

(Doplnenie ciest je posledný krok fázy 1 — Príprava dát, viď prednáška 8 — schéma fáz procesu WUM.)

Úloha 01: doplnenie_stt_mean.py

Teoretický základ: Podľa prednášky 8 je sedenie sekvencia navštívených stránok s časovou známkou, pre ktorú platí podmienka na časový rozdiel `tk − tk−1 ≤ δ`. Keď je odhad časového okna `δ` určený **priemerom dĺžky sedenia** (STT_Mean), heuristika doplnenia funguje **iba v rámci jedného sedenia** — medzi sedeniami nemá navigačný význam vkladať medzihop, lebo časová známka by porušila definíciu sedenia. *(viď prednáška 8 — Identifikácia sedení na základe času.)*

Čo robí: Doplní chýbajúce medzistránky do logu, ktorého sedenia boli identifikované metódou STT_Mean (časový prah = priemer dĺžky sedenia).

Vstup → výstup: - `in/cleaned_log4_stt_mean.csv` → log po STT_Mean (USER_ID, SESSION_ID, UNIXTIME, URL, IP...) - `in/mapa_webu.xlsx` → strom webu UKF - `out/cleaned_log4_doplnene.csv` → log + ~750 doplnených záznamov

Kľúčové kroky: 1. Načítaj log (`pd.read_csv`) 2. Načítaj mapu (`pd.read_excel`), premenuj stĺpce, zahoď 1. riadok 3. Skonvertuj URL → cesty (odstráň `https://www.ukf.sk`) 4. Normalizuj prázdny root na `/` 5. Postav `parent = dict(zip(path, ref_path))` a `map_paths = set(...)` 6. Iteruj pármi (curr, next_row), doplň ref_page ak chýba zdroj ale cieľ je v mape 7. Ulož cez `pd.DataFrame(new_rows).to_csv(..., index=False)`

Dôležité parametre: - `regex=False` v `str.replace` — doslovná substitúcia, rýchlejšie + bezpečnejšie pre bodky v doméne - `.iloc[1:].reset_index(drop=True)` — drop=True znamená že starý index NEPRIDAJ ako stĺpec - `index=False` v `to_csv` — bez RangeIndex (0,1,2...) v CSV

Vzorec / formula: - STT_Mean prah = priemer dĺžky sedenia (Length)

Časté otázky pri obhajobe: - **Q:** Prečo dedí doplnený záznam UNIXTIME z curr? **A:** `new_row = curr.to_dict()` , prepíše sa len URL. Pri `diff(UNIXTIME)` dáva 0 — drobný edge case. - **Q:** Prečo doplnenie len v rámci sedenia? **A:** Medzi rôznymi sedeniami nie je navigačné prepojenie — rôzne časové okno, typicky iný používateľ/prestávka. *(viď prednáška 8 — definícia sedenia cez časovú podmienku.)*

Úloha 02: doplnenie_stt_q.py

Teoretický základ: Prednáška 8 hovorí: *„Ak premenná Length nemá exponenciálne rozdelenie, na odhad časového okna δ môžeme použiť kvartilový odhad* `δ = Q3 + 1.5 × IQR`." Pre dáta s ťažkým chvostom je priemer skreslený outliermi, preto použitie odhadu strednej hodnoty **nie je vhodné**. STT_Q teda dáva robustnejší prah a sessionizovaný log slúži ako vstup pre identickú heuristiku doplnenia. *(viď prednáška 8 — kvartilový odhad časového okna.)*

Čo robí: Doplní cesty do logu, kde sedenia boli identifikované STT_Q (kvartilový prah pre neexponenciálne dáta).

Vstup → výstup: - `in/cleaned_log4_stt_q.csv` (~20667 riadkov) - `in/mapa_webu.xlsx` - `out/cleaned_log4_doplnene.csv` (~21402 riadkov, 735 doplnených)

Kľúčové kroky: 1. Identická logika ako úloha 01, ale vstup je STT_Q sessionizovaný log 2. Konštrukcia parent dict + map_paths set 3. Iterácia, podmienka `url_from ∉ map AND url_to ∈ map` 4. Doplnenie ref_page ako medzihop

Dôležité parametre: - `set` namiesto `list` pre map_paths → O(1) vs O(n) pri 21k iteráciách = radové zrýchlenie - `list.append + pd.DataFrame na konci` namiesto `df.append` v cykle → O(n) vs $O(n^2)$ - `.get(key)` namiesto `[key]` → vráti None bez KeyError

Vzorec / formula: - STT_Q prah = `Q3 + 1.5 × IQR` (75. percentil + 1.5 × kvartilové rozpätie — mierne outliery v boxplote)

Časté otázky pri obhajobe: - **Q:** Prečo STT_Q a nie STT_Mean? **A:** Pre neexponenciálne rozdelené Length sa odhad priemeru **nepoužíva** — používa sa kvartilový prah, ktorý je odolný voči outlierom. *(viď prednáška 8 — „odhady strednej hodnoty sa síce používajú, ale ich použitie nie je vhodné".)* - **Q:** Čo je audit-finding 'iba 1 prípad'? **A:** Heuristika rieši **iba** prípad „url_from chýba v mape, url_to je v mape". Prípad „obe v mape ale nesusedné" nepokrýva (mierne nižšie počty doplnených).

Úloha 03: doplnenie_rlength.py

Teoretický základ: Podľa prednášky 8, ak premenná Length **má exponenciálne rozdelenie** (`f(x) = λe^(-λx)`, `F(x) = 1 - e^(-λx)`), na odhad časového okna sa použije **kvantilová funkcia** `δ = -ln(1 - p) / λ`, kde `1/λ` je priemer dĺžky návštev a `p` je relatívna početnosť navigačných stránok (`0 ≤ p < 1`). Typická voľba `p = 0.4` znamená, že 40 % najkratších hodnôt sa považuje za navigačné kliky a delí ich od obsahových stránok. *(viď prednáška 8 — kvantilová funkcia exponenciálneho rozdelenia, MLE odhad `λ = 1/x̄`.)*

Čo robí: Doplní cesty pre log so sedeniami RLength (exponenciálny prah).

Vstup → výstup: - `in/cleaned_log4_rlength.csv` → log po RLength - `in/mapa_webu.xlsx` - `out/cleaned_log4_doplnene.csv` → 776 doplnených (očakávaných ~850)

Kľúčové kroky: 1. Načítanie logu (už má SESSION_ID z RLength) 2. Mapu webu spracovať na cesty + parent dict 3. Iterácia pármi v rovnakom sedení 4. Doplnenie parent(url_to) ako medzihop

Dôležité parametre: - `dict(zip(path, ref_path))` — pri duplicitách vyhrá **posledný** záznam - `ref_page != url_from` — zabráni cyklu „from → from → to"

Vzorec / formula: - RLength: `-ln(1 - p) × mean` (kvantilová funkcia exp. rozdelenia) - Pre `p = 0.4`: 40 % hodnôt = navigačné stránky (krátke kliky)

Časté otázky pri obhajobe: - **Q:** Prečo 776 < 850? **A:** Heuristika nepokrýva „oba v mape ale nesusedné" + audit-finding `.iloc[1:]` zahodí 1 reálny záznam mapy. Akceptovateľné ~9 % pod cieľom. - **Q:** Prečo dopĺňa len 1 medzistránku? **A:** Pre viacúrovňové medzery (`/a → /a/b/c`) by korektný algoritmus rekonštruoval celý reťazec medzihopov *(viď prednáška 8 — príklad mapy webu, kde sa pri jednom skoku dopĺňa celá cesta A→B→C→D)*. Tu sa to zjednodušuje na jeden krok.

Úloha 04: doplnenie_href.py

Teoretický základ: Prednáška 8 ukazuje, že okrem časovej podmienky existuje aj **štruktúrna podmienka** založená na odkazujúcej stránke (Referrer). Ak je odkazujúca stránka **dostupná** (Extended Log File), sedenie je definované postupnosťou, v ktorej `Referrer(pi) = pi-1`. Ak rovnosť neplatí (alebo Referrer nie je definovaný, t. j. `= "-"`), záznamy patria do dvoch rozdielnych sedení. Metóda **h-ref** kombinuje STT_Mean s touto kontextovou podmienkou — priamy prístup zvonku (`Referrer = "-"`) štartuje nové sedenie. *(viď prednáška 8 — identifikácia sedení podľa štruktúry/času, formálna definícia z = (p1, t1), ..., (pn, tn).)*

Čo robí: Doplní cesty pre log so sedeniami h-ref (STT_Mean + kontextová podmienka `Referrer = "-"`).

Vstup → výstup: - `in/cleaned_log4_href.csv` → log po h-ref - `in/mapa_webu.xlsx` - `out/cleaned_log4_doplnene.csv` → ~690 doplnených

Kľúčové kroky: 1. Načítanie logu (h-ref sessionizácia z cv2/09) 2. Mapa webu → parent dict + map_paths set 3. Iterácia v rámci sedenia, h-ref doplnenie ref_page

Dôležité parametre: - Rovnaké ako úlohy 01–03 — algoritmus je identický, mení sa **iba** vstupný súbor

Vzorec / formula: - h-ref = STT_Mean prah + (`Referrer == '-'` → nové sedenie)

Časté otázky pri obhajobe: - **Q:** Prečo h-ref má menej doplnení (690) než RLength (850)? **A:** h-ref má **viac** sedení (46k vs 33k v projekte) ale **kratších**. Krátke sedenia majú málo skokov → menej čo doplniť. - **Q:** Ako sa líši h-ref od STT_Mean? **A:** STT_Mean = iba časový prah. h-ref = STT_Mean + 3. podmienka na Referrer (priamy prístup zvonku = nové sedenie). Štruktúrna podmienka pochádza z definície sedenia cez referer v prednáške 8.

🎯 Vyhodnotenie / na čo si dať pozor pri obhajobe

- **Path Completion = rekonštrukcia chýbajúcich stránok** z mapy webu, **nie** pridávanie nových stránok

- **Mapa webu = strom** s `ReferencePage` (rodič) pre každú URL — postavená z najčastejšieho referrera *(viď prednáška 8 — príklad „aktuálna navigácia vs. mapa webu")*
- **Doplnenie len v rámci sedenia** — cez hranice je nezmyselné, lebo by porušilo časovú podmienku sedenia
- **Iba 1 medzistránka medzi párom** — viacúrovňové rekonštrukcie (A→B→C→D) algoritmus nerieši, hoci prednáška 8 ukazuje ideálny prípad s plnou rekonštrukciou
- **Heuristika pokrýva iba prípad „url_from chýba, url_to je v mape"** — „oba v mape ale nesusedné" sa nedopĺňa
- **Doplnené stránky zlepšujú kvalitu pre fázu 2 WUM** (Apriori, sekvenčné pravidlá) — bez nich by pravidlá mali „diery"
- **Doplnený záznam dedí UNIXTIME predchodcu** → diff = 0 (mierny edge case pre analýzu zotrvania)
- **Doplnenie ciest je posledný krok fázy „Príprava dát"** v CRISP-DM podľa prednášky 8 — bez neho by pattern discovery (Apriori, PrefixSpan) operoval nad neúplnými cestami